

EXP6

April 10, 2026

```
[1]: import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.svm import SVC
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV
```

```
[5]: data = load_breast_cancer()
X = data.data
y = data.target

print(X.shape)
```

(569, 30)

```
[7]: scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
[9]: pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

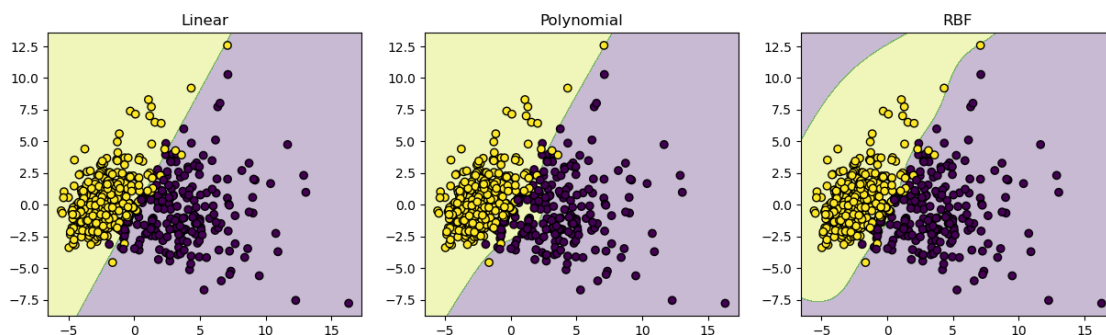
```
[11]: X_train, X_test, y_train, y_test = train_test_split(
    X_pca, y, test_size=0.2, random_state=42
)
```

```
[13]: models = {
    "Linear": SVC(kernel='linear', C=1),
    "Polynomial": SVC(kernel='poly', degree=3, C=1),
    "RBF": SVC(kernel='rbf', gamma='scale', C=1)
}

for name, model in models.items():
    model.fit(X_train, y_train)
    acc = model.score(X_test, y_test)
    print(f"{name} Kernel Accuracy: {acc:.2f}")
```

Linear Kernel Accuracy: 0.99
Polynomial Kernel Accuracy: 0.89
RBF Kernel Accuracy: 0.96

```
[16]: def plot_decision_boundary(model, X, y, title):  
    h = 0.02  
    x_min, x_max = X[:,0].min() - 1, X[:,0].max() + 1  
    y_min, y_max = X[:,1].min() - 1, X[:,1].max() + 1  
  
    xx, yy = np.meshgrid(  
        np.arange(x_min, x_max, h),  
        np.arange(y_min, y_max, h)  
    )  
  
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])  
    Z = Z.reshape(xx.shape)  
  
    plt.contourf(xx, yy, Z, alpha=0.3)  
    plt.scatter(X[:,0], X[:,1], c=y, edgecolors='k')  
    plt.title(title)  
  
plt.figure(figsize=(15,4))  
  
for i, (name, model) in enumerate(models.items()):  
    plt.subplot(1,3,i+1)  
    plot_decision_boundary(model, X_pca, y, name)  
  
plt.show()
```



```
[18]: param_grid = {  
    'C': [0.1, 1, 10],  
    'gamma': [0.01, 0.1, 1],  
    'kernel': ['rbf']  
}
```

```
grid = GridSearchCV(SVC(), param_grid, cv=5)
grid.fit(X_train, y_train)
```

```
print("Best Parameters:", grid.best_params_)
print("Best Score:", grid.best_score_)
```

```
Best Parameters: {'C': 10, 'gamma': 0.01, 'kernel': 'rbf'}
Best Score: 0.9340659340659341
```

```
[ ]:
```